PROJECT OF CP1010: THE DATA APP

For this project, I chose to build upon the foundational **p5.js Data Visualization template**. My goal was to create a dual-topic interactive dashboard tracking two distinct themes that capture the attention of Generation Z today: the environmental realities of Climate Change and the booming digital media economy of Virtual YouTubers (VTubers). The application acts as a clean, unified platform designed to educate users on critical climate data while normalizing and highlighting the massive global footprint of virtual content creation.

To transform the basic template into a multi-layered application, I built a custom, canvas-based routing system. For the Climate Change category, the app branches out into five distinct visualization extensions:

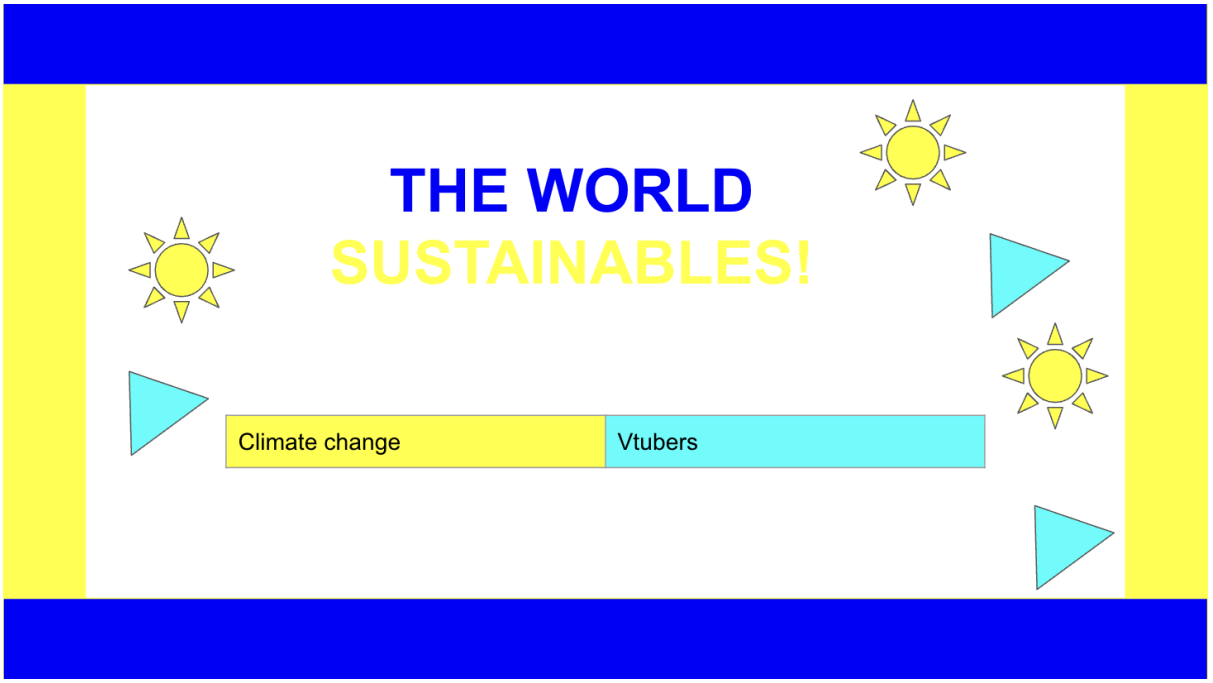
- An interactive time-series chart mapping temperature anomalies against global CO2 spikes.
- A comprehensive breakdown of modern renewable energy distribution.
- Comparative bar charts exploring agricultural impacts and food waste.
- An educational text dashboard showcasing alternative, sustainable farming methods.
- A data-driven asset panel contrasting emissions from 3D printing against traditional manufacturing.

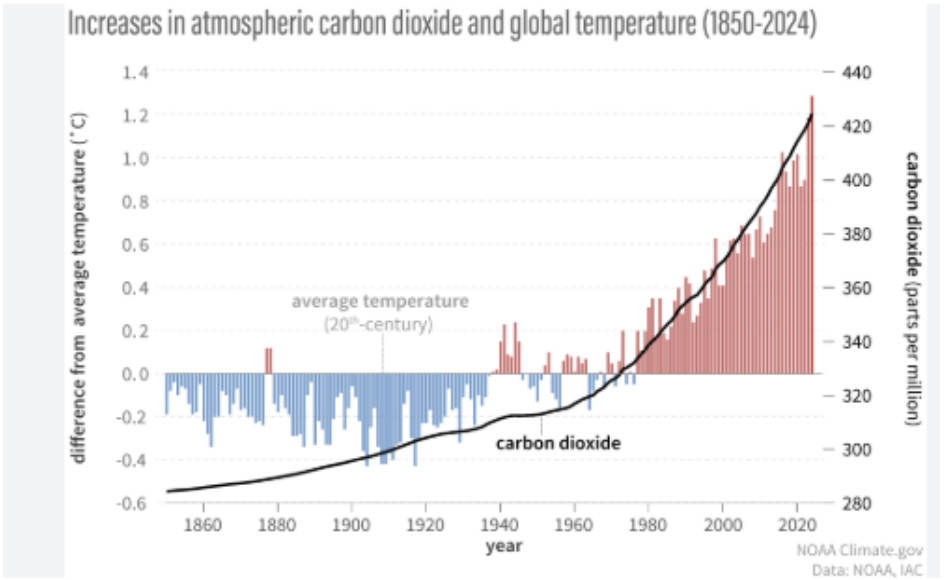
For the VTuber hub, the template expands into three unique presentation screens: a top-10 global subscriber distribution bar chart, a live superchat revenue leaderboard, and an interactive structural information breakdown.

From a technical perspective, the entire program relies heavily on modular programming and Object-Oriented design. Rather than cluttering a single file or depending on standard HTML browser elements (like DOM sidebars or buttons), every single visualization is isolated as a self-contained script class containing uniform lifecycles. These independent modules are managed globally by a custom Gallery container constructor function. Flat arrays dynamically feed the raw datasets into the program, while custom coordinate mapping algorithms handle canvas offsets, scale the bar charts programmatically, and manage color transitions on our 1024x576 workspace. A central state-machine variable (currentScreen) monitors user input to seamlessly transition between menus.

The most challenging hurdle during development was engineering a reliable canvas-click routing algorithm without overlapping visual components. When breaking away from basic browser DOM buttons, reading mouse clicks natively inside a p5.js loop meant defining strict, manual pixel bounding boxes. Early on, structural indexing mismatch errors caused a major bug where clicking buttons on active climate charts would accidentally route the user deep into the VTuber sub-menus. I resolved this by eliminating automatic linear loops inside

gallery.js and binding the shrunken, top-left BACK hitboxes to isolated state conditions that strictly verify the active category before executing a screen change.

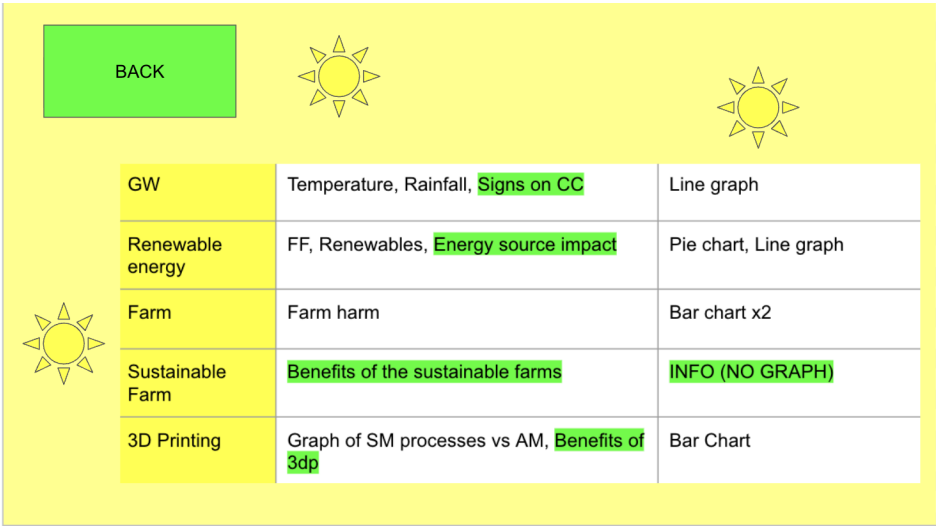




My current design work reflects a complete transition from standard web browser layouts into an entirely immersive, custom-drawn p5.js canvas environment. During the initial brainstorming and interface sketching phases, I originally planned to use native HTML sidebar panels and DOM button tabs to handle category selection. However, as the project evolved, I realized that mixing browser DOM styles with p5.js canvas loops disrupted the fluid visual scaling and corrupted my state-routing variables. To fix this, I completely scrapped the HTML sidebar elements and shifted to a standalone canvas system where all graphical interfaces, menus, and layout accents are rendered directly by code.

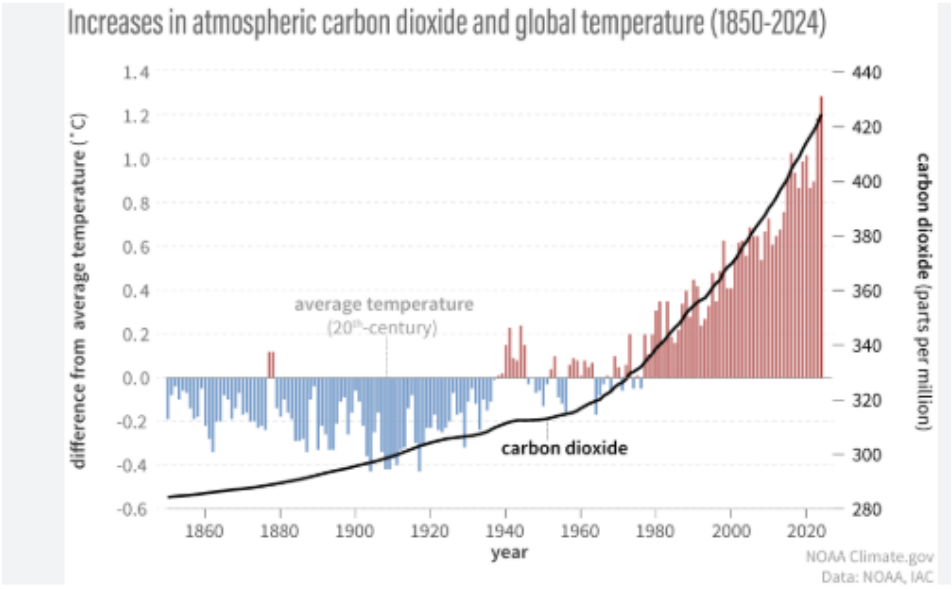
The visual design of the home screen introduces a balanced workspace framed by deep blue geometric borders and light blue play-button icons. This layout presents two large, central interactive click zones: one dedicated to Climate Change and the other to VTubers. The color palette and ambient decorative choices for each sub-menu were intentionally selected to mirror the core concepts of their respective topics:

- **Climate Change Hub:** This sub-menu features a warm yellow background decorated with mini sun sprites. This visual styling represents atmospheric heat trapping while simultaneously highlighting how solar light serves as a clean alternative to fossil fuels. The five internal screens map out specific climate realities:

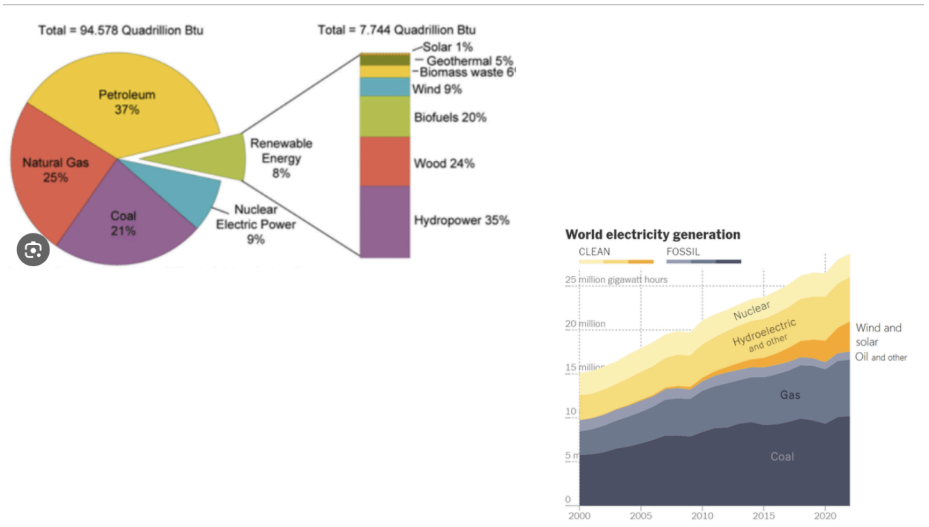


GW	Temperature, Rainfall, Signs on CC	Line graph
Renewable energy	FF, Renewables, Energy source impact	Pie chart, Line graph
Farm	Farm harm	Bar chart x2
Sustainable Farm	Benefits of the sustainable farms	INFO (NO GRAPH)
3D Printing	Graph of SM processes vs AM, Benefits of 3dp	Bar Chart

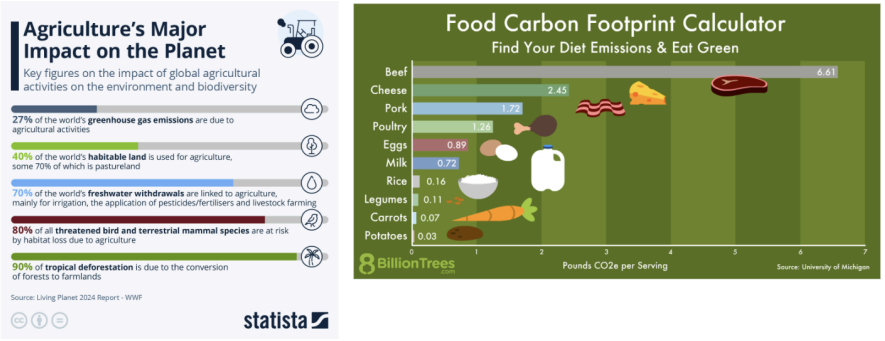
a dual time-series graph tracking global temperature anomalies alongside rising CO2 parts per million based on NOAA data trends,



a unified chart tracking clean energy distribution against fossil fuel peaks,

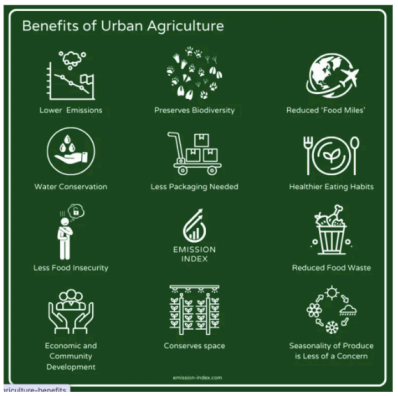


agricultural impact bar charts comparing food production emissions,

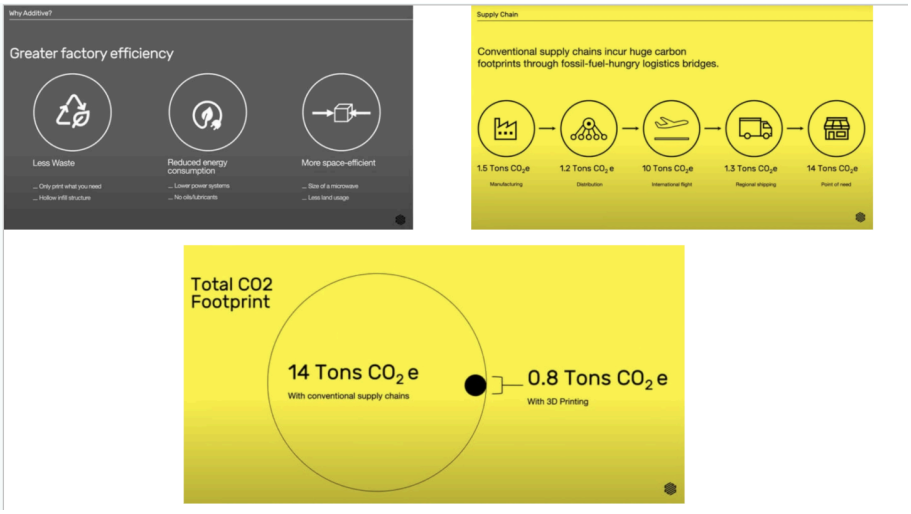


a text-based alternate farming module,

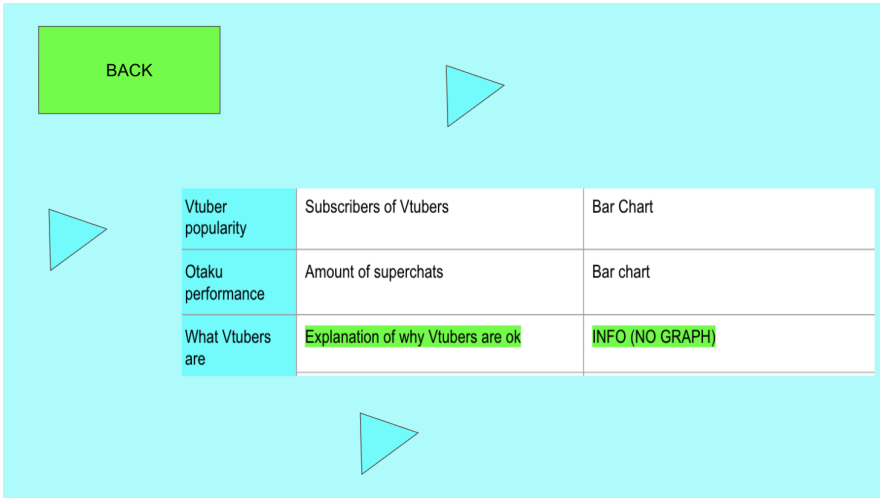




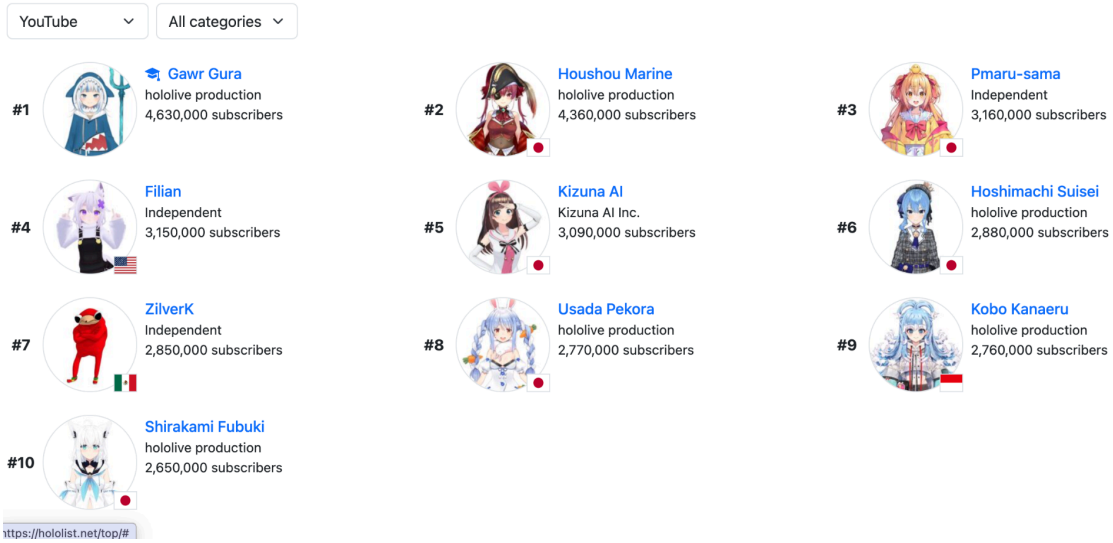
and a comparison model illustrating the carbon-saving advantages of 3D printing.



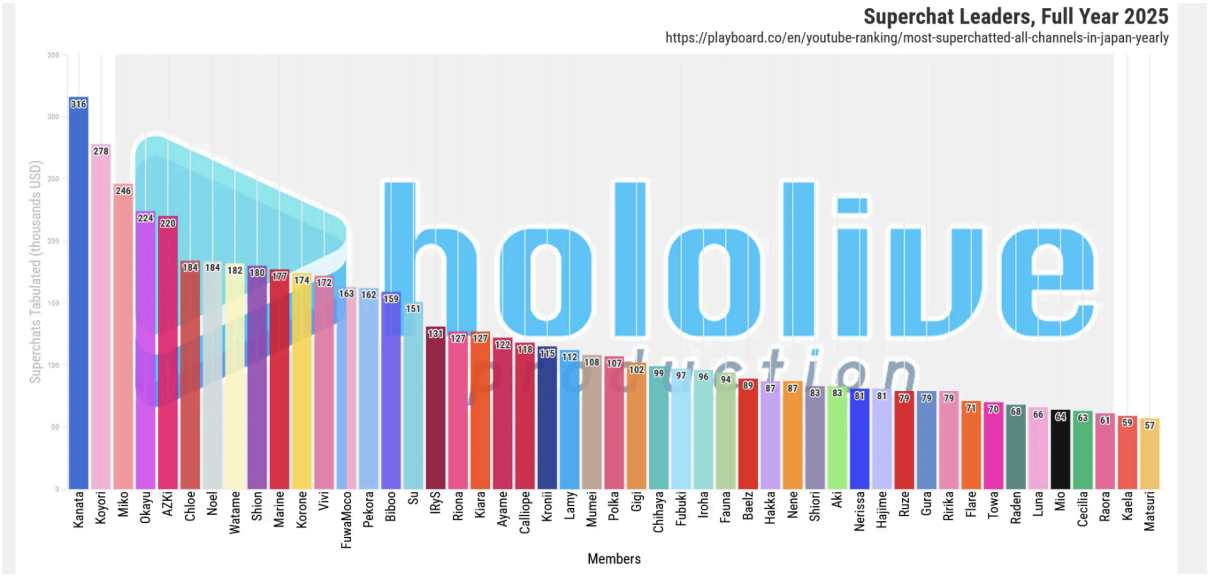
- **VTuber Hub:** This sub-menu transitions to a clean, light blue background accented with custom geometric play icons inspired directly by the prominent Hololive media ecosystem. The interface provides three interactive tracking views based on live digital media analytics.



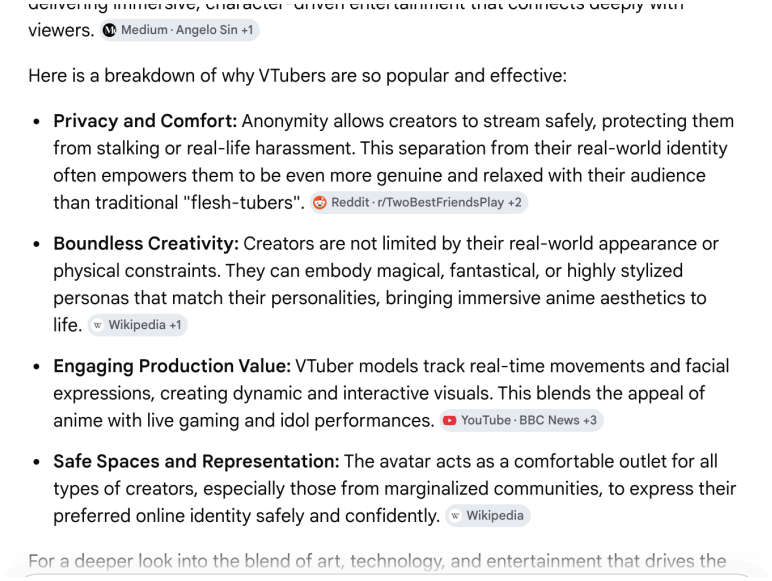
The subscriber distribution chart scales data for the top ten global channels (tracking creator milestones like Gawr Gura's 4.63M subscriber peak),



the revenue chart visualizes global Superchat donation leaders using annualized metrics from Playboard.co,



and the final information panel summarizes the privacy benefits and creative security that drive the virtual talent space.



In terms of code progress, the core architecture is entirely finalized and stable. The central Gallery manager successfully initializes all eight sub-visualization classes. Every dashboard screen features a functional, programmatically bounded BACK button that seamlessly reverts the application state back to its parent category menu. All external text data files, coordinate scaling matrices, and color systems have been fully integrated, ensuring the application operates as a responsive, polished, and unified standalone program.

To manage development across the full duration of the project, I used Agile project management principles to prioritize my coding workflow. I estimated a total commitment of roughly 40 hours to build, test, and document the application. Rather than working linearly, I followed the Agile method by tackling the most complex, "must-have" features first before moving on to minor aesthetic enhancements.

I allocated my time in this specific order because a data visualizer cannot function without a stable environment. I spent the first two weeks entirely on the core infrastructure: engineering the main state-routing engine, building the modular `Gallery` container, and managing canvas click logic. Once the structural routing was working perfectly, I spent week three binding the flat datasets to my bar charts and time-series line graphs. Only after the charts were rendering accurately did I dedicate week four to "nice-to-have" features, such as drawing the custom yellow suns and blue play-button icon assets. Finally, I allocated the last week as a dedicated time buffer to handle coordinate debugging and write this report.

Below is the synchronized timeline table detailing how my development schedule matches my project Gantt chart:

Activity Description	Start Date	End Date	Duration	Agile Classification
Brainstorming & Interface Layout Sketches	Apr 15	Apr 30	15 Days	Core Design Phase
State Machine Engine & Gallery Constructor	May 1	May 31	31 Days	High-Difficulty / Must-Have
Data Binding & Chart Coordinate Mapping	Jun 1	Jun 30	30 Days	Technical Logic / Must-Have
Aesthetic Themes & Custom Canvas Assets	Jul 1	Jul 20	20 Days	Visual Polish / Nice-to-Have
Buffer Period: Coordinate Debugging & Reports	Jul 21	Aug 10	20 Days	Risk Mitigation Buffer

1. Primary Data & Analytical Sources

- **National Oceanic and Atmospheric Administration (NOAA) & NASA:** Historical time-series datasets (1850–2024) used to accurately plot global temperature anomalies against rising atmospheric carbon dioxide (CO_2) parts per million.
- **Playboard.co:** Annualized global YouTube database archives used to compile real-world Superchat monetization metrics and ranking leaderboards for top-performing virtual creators.
- **Hololive Production (Cover Corp):** Visual branding guidelines and public community media kits that inspired the light blue color palette and geometric play-button icon assets.
- **Statista & University of Michigan (Food Carbon Footprint Calculator):** Industry greenhouse gas emissions audits utilized to map comparative bar charts for global agricultural impacts, food waste, and traditional versus additive (3D printing) manufacturing supply chains.

2. Software Frameworks & Libraries

- **p5.js Library (p5.min.js):** The primary open-source JavaScript framework used to render the 1024x576 graphical canvas workspace, process real-time drawing loops, and manage native vector shapes.
- **p5.dom Library (p5.dom.min.js):** Used during early structural prototyping stages to test interaction variables prior to building the pure canvas navigation engine.
- **Coursework Visualizer Base Template:** The structural framework provided in the course materials from which this interactive multi-tier dashboard extension was built and customized.